

Specification: Autonomous Local LLM System

Optimized for Environment: GPU VRAM 8GB / RAM 32GB / WSL2 (Rocky Linux 10)

1. System Objectives

The primary objective of this architecture is to build a completely local, self-contained AI Software Development Lifecycle (SDLC) loop on a standard consumer PC. By effectively segregating workloads between hardware assets, this system achieves professional-grade planning, coding, reviewing, and security auditing without leaking data or incurring external API costs.

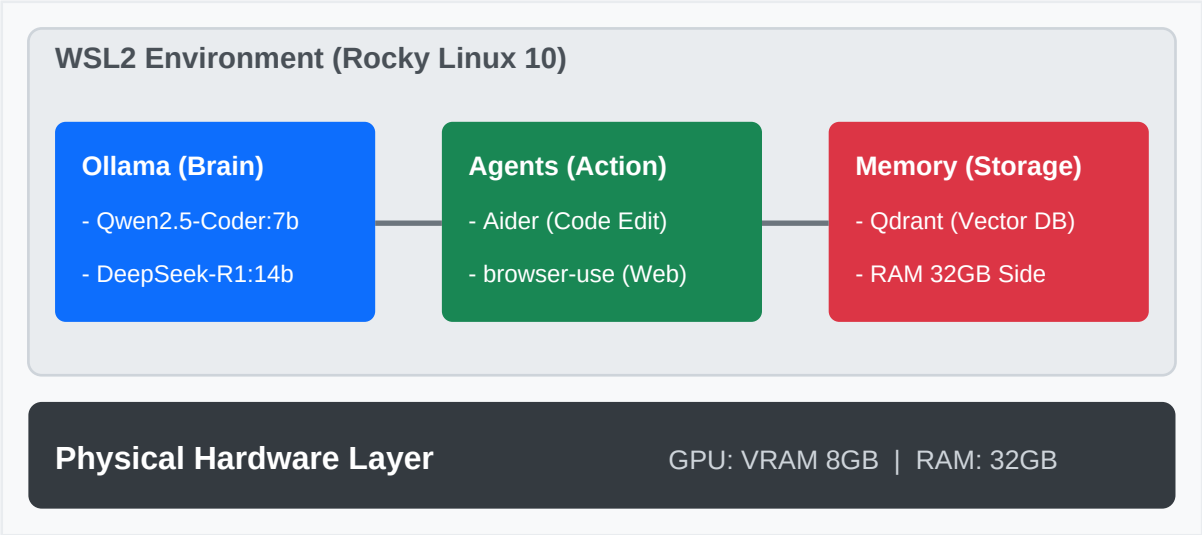
Key Targets:

- **Zero-Cost Operation:** Run elite development models natively without cloud dependency.
- **Self-Evolution Loop:** Empower agents to autonomously browse technical documentation, update local knowledge, write source files, execute test suites, and audit errors.
- **Resource Maximization:** Overcome VRAM limitations using dynamic quantization and unified system memory routing.

2. Core Software Matrix

Category	Software Components	VRAM/RAM Strategy	Primary Role in Loop
OS/Runtime	WSL2 (Rocky Linux 10) + Docker Desktop	System RAM managed by host OS	Provides low-latency POSIX execution layer and sand
AI Brain	Ollama - Qwen2.5-Coder:7b - DeepSeek-R1:14b	7b: Fits fully in 8GB VRAM 14b: Spills safely into 32GB RAM	7b handles continuous rapid coding. 14b manages deep planning and security audits.
Action Layer	Aider + browser-use (Playwright)	Low VRAM overhead (Text and browser processes)	Aider modifies raw files via git diff tools. browser-use navigates SPA sites autonomously.
Knowledge DB	Qdrant (Vector DB) + bge-m3 Embedding	100% Routed to 32GB RAM (Offloads GPU completely)	Chronicles learned web documentation, error fixes, an

3. Local Architecture Diagram



4. Operational Production Scenario (Example: Hotfix Scenario)

Below is the exact execution timeline showing how the system acts autonomously when an application bug is encountered in production:

Phase A: Self-Directed Research & Analysis

1. The developer passes a raw error signature to the Agent Core: *'Application crash on concurrent DB pool requests.'*
2. **browser-use** spins up a headless Chromium instance inside Rocky Linux. It automatically queries modern tech boards, navigates JS-heavy documentation frameworks, and extracts current workarounds for the connection pool bug.
3. The extracted document chunks are vectorized using **bge-m3** entirely on the system CPU and committed to the local **Qdrant** database.

Phase B: Autogenous Planning & Implementation

4. **DeepSeek-R1 (14b)** pulls the solution context from Qdrant, reviews the project's system design, and formulates a multi-step execution plan to rewrite the pool lifecycle manager.
5. The pipeline shifts to **Qwen2.5-Coder (7b)** inside VRAM for lightning-fast text processing. **Aider** targets the vulnerable local files, using specialized AST tools to make surgical edits without disturbing surrounding codebase logic.

Phase C: Automated Auditing & Git Lifecycle

6. Aider triggers the local test suite via the terminal. If a regression occurs, the 14b reasoning model intercepts the stdout log, corrects the planning path, and instructs the 7b model to apply a patch.
7. Once all tests achieve 100% compliance, the agent writes a concise, professional commit message and executes a native *git commit*.